

Leveraging AI and Synthetic Data in TAF-based Research

Becky Staiger
University of California, Berkeley

Medicaid Data Learning Network
September 15, 2026

Overview

- What we want to do: use Medicaid (and Medicare) administrative claims for research
- Problem: this is often hard to do
 - Expensive!
 - Getting access takes a long time
 - Secure servers with data may be “clunky,” often don’t have internet
- Goal: become more efficient — higher quality work in a reasonable amount of time

The proposal: synthetic data

Build **synthetic data** to train the code on

- Mirror the real files — structure, naming, defects, moments
- Write and test the whole pipeline locally, with full tooling
- Transfer in code that already works

Today: a **blueprint** for generating synthetic MAX + TAF data

- GitHub repo:
<https://github.com/beckystaiger87/synthetic-mcaid-claims-blueprint>
- You generate your own synthetic data, for your states, years, files and question.
- Why not make synthetic data available?
 - What I need in my project may not be relevant to you (and vice versa)
 - To be useful, would take up too much space

Note: this talk is built around access via VRDC, but is equally relevant for physical-file access

How the synthetic data is built

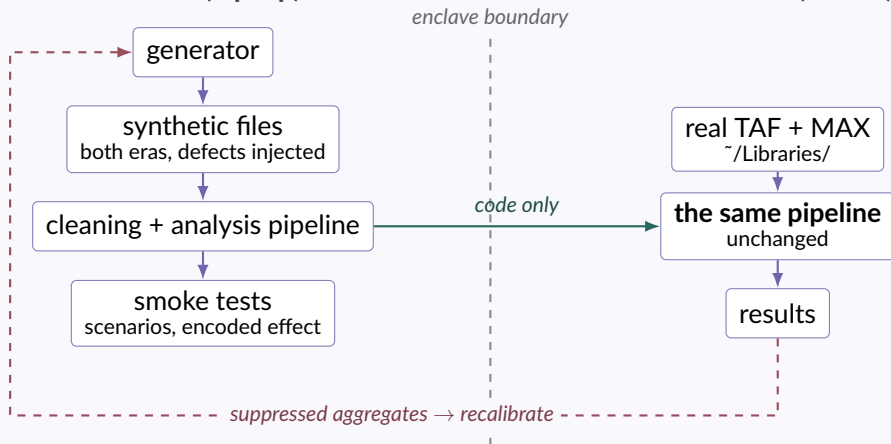
The shape of it

AI assistance OK · no restricted data

LOCAL (laptop)

no internet · no AI · gated output

ENCLAVE (VRDC)



One direction carries code. The other carries only suppressed aggregates.

Six ingredients

	Ingredient	What it contributes
1	Variable names from public data dictionaries	the synthetic columns <i>are</i> the real columns; a check flags any that drift
2	Mirror the file structure	header/line splits, 12 monthly columns, slot×month grids, two person ids
3	Harmonize eras	one pipeline reads both MAX and TAF
4	Inject known data-quality issues	the cleaner has something to catch – clean data tests nothing
5	Calibrate to real moments	public sources first, then suppression-compliant CMS aggregates
6	Encode truths about the DGP	and test that your analysis code recovers them

On **1**: dictionaries are public, but naming differs by vintage and by provider. The check flags a mismatch; **you fix it by hand against the files you actually have.**

More details on 4, 5 and 6.

4 — inject the defects

Defect	Cause	Rate	Caught by
Missing servicing NPI	LPI sent instead	5–25%	xwalk, then drop blanks
Off-year service dates	late-filed claims	~1%↑	year from the date column
Exact duplicate rows	resubmissions	~1%↑	unique() on natural key
Duplicate bene-years on PS/DE	cross-state movers; enrollment corrections	~0.5%	drop all rows for that bene-year
Unlinked BENE_ID	CCW linkage failed	1–5%	fallback key
BENE_ID reassigned	MAX → TAF	~5%	era crosswalk
Mixed NDC length	11 vs 13 char	~10%	normalize on read
Constant sentinel	one plan's placeholder	1 plan	tabulate by submitter

↑ = injected *above* the real rate, so a small sample still shows it. Record that next to the parameter:

```
offyear_date_rate = 0.015, # ABOVE REAL (~0.3%) for detectability
```

Without the comment, someone later “corrects” it to the true rate and switches the test off.

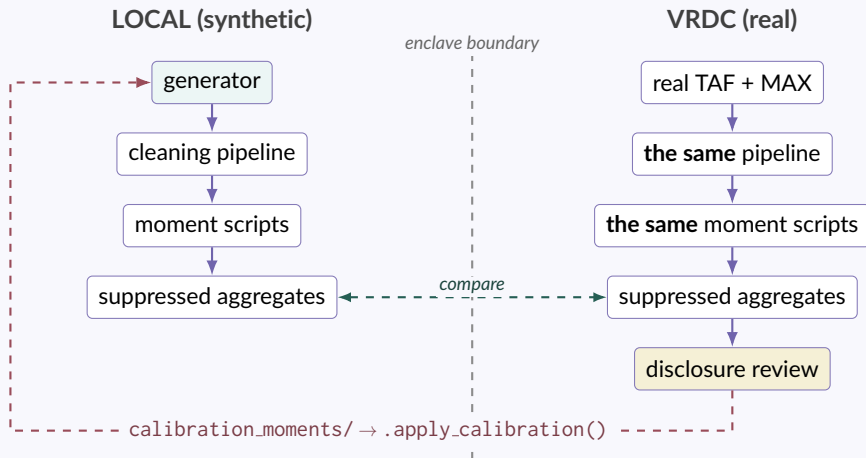
5 — calibrate to real moments: public data

Public source	Calibrates
MBES / CMS-64	enrollment by state-year
DQ Atlas	injection rates; which state-years are unusable
KFF state summaries	eligibility-group mix
NPPES	a real provider universe

All publicly available, unrestricted information.

Note: the NPPES is useful in generating synthetic data with real NPI's. This can be used to reproduce a match rate you'd actually face in the data, should you match on other NPI-identified datasets.

5 — calibrate to real moments: server data



6 — encode truths about the data-generating process

- Set a “**known**” effect in the generator — e.g. losing your usual provider reduces the probability of an E&M visit in a given month by **15 percentage points** ($0.36 \rightarrow 0.21$)
- Say what unit it lives at: here, an **enrolled bene-month**
- Write it to a **truth file**. *Analysis code never reads it*
- Run your estimator. Make sure you can recover this estimate.

The setting is from Staiger (2022), *Disruptions to the Patient-Provider Relationship and Patient Utilization and Outcomes*, J. Health Econ. 81:102574. The estimated effect in the paper is -3.7pp on a 0.679 pre-exit base, per bene-quarter; the magnitude encoded here is larger for sake of illustration.

One estimator, two denominators

```
$ Rscript example/run_all.R
```

```
[bene-month] estimate: -0.1425 (SE 0.0050) CI [-0.1523, -0.1327]
```

```
encoded: -0.1500
```

```
PASS - CI covers the encoded effect.
```

```
[claims-file months] estimate: -0.1667 (SE 0.0076) CI [-0.1817, -0.1518]
```

```
encoded: -0.1500
```

```
rows: 91,998 of 207,694 enrolled bene-months (56% of the panel is gone)
```

```
FAIL - off by 11%.
```

- The effect is a probability **per enrolled bene-month** — so months with **no claim** belong in the denominator
- Build the panel from the **claims** file and those months are simply absent
- Both run without error · both significant · both correctly signed

Losing your provider shrinks the outcome *and* the sample.

Workflow, and where AI fits

Thought partner, not autonomous agent

- **Start from a puzzle, not a coding request.**

“I’m seeing Y — what could explain it?” produces understanding. “Write me a query that does X” produces code that may answer the wrong question.

- **The agent has amnesia.**

Every session starts ignorant. The markdown files *are* the institutional memory — documentation is an output, not an afterthought.

- **Verification through redundancy.**

Two implementations agreeing is evidence. One running without error is none.

- **Adversarial review requires separation.**

The instance that wrote the code cannot audit it.

- **Legible or unchecked.**

A skimmed result is an unverified result — so how a finding is written is part of whether it gets verified at all.

Harness: Scott Cunningham’s `mixtapetools`, adapted for enclave constraints

Check it as you build it

Every construction step writes a **QC report** next to its output. Committed, not scratch.

```
# QC | 02_pull_ot                                0.5s · FLAG (1)

OK      de-dup removes under 5% of lines          1.0%
OK      crosswalk resolved some legacy ids        13,138 resolved
OK      under 15% dropped, unresolvable servicing  7.5%
OK      MAX carries no billing NPI
FLAG    no plan reports a single billing NPI      CA0000002
```

- **Catches a nonsensical number now**, while you still remember what you changed
- **Records the intermediate**. Six weeks on, “when did this change?” has an answer

Three checks

	QC per step	Blindspot	Referee 2
Asks	does this number make sense?	can you see what's in front of you?	is this implemented correctly?
When	as you write the step	output appears, <i>before</i> interpreting	end of an arc
Session	inline	same — proximity is the point	fresh — independence is the point
Catches	rows lost without a reason	the unexplained spike, the check nobody ran	join fan-out, clustering, disclosure
Cost	seconds	minutes	hours

	QC	Blindspot	Referee 2
A diagnostic you will cite anywhere	run	run	—
A step other steps depend on	run	run	at arc end
Anything feeding a headline number	run	run	run

Do not follow the last two blindly. A generated list of concerns is a list of *candidates*, and not all are worthwhile to implement.

Working example

Project question: how does physician practice ownership determine a physician's Medicaid participation? MAX/TAF 2012 – 2022, three specialties, event studies leveraging acquisition dates

- Team members: 4 – 2 of us working with the data, 1 VRDC seat
- Create synthetic data
- Train code on the synthetic data (SAS, R)
- Export NPI-level, suppression-compliant measures of provider participation; link to our existing ownership database (from this [paper](#))
- Conduct analyses outside of server to troubleshoot (get 99% of the way there)
- At the end: run in the VRDC

Limitations

- **Most real bugs are invisible here.** A filter, not a proof
- **Only the defects you already know about.** Real data still surprises you
- **The numbers mean nothing.** A test harness, not data
- **Validates code, not design.** Parallel trends hold because you built them in
- **It rots.** A stale generator breaks pipeline code in confusing ways
- **Calibration needs server access.** You cannot do it before you are in

It buys the cheap bugs, so server time goes to the expensive ones.

Take-aways

Who might use this

A research team — especially when only one member has a VRDC seat

- everyone else writes and tests against synthetic files
- the seat-holder runs finished code instead of debugging it
- no waiting for a second seat to be approved and paid for

Students

- learn what a TAF pipeline looks like with no DUA
- a real task with a checkable answer: recover the encoded effect
- productive during the 6–18 months access takes

And anyone who wants their pipeline checked.

Nobody can rerun a TAF analysis without a DUA — so nobody does. Anyone can run the generator plus the pipeline.

Resources

These slides and the synthetic-data blueprint

github.com/beckystaiger87/synthetic-mcaid-claims-blueprint

SYNTHETIC- DATA-BLUEPRINT.md	the specification: six ingredients, reader contract, build order
R/ references/ docs/ templates/	column registry, validator, defect injectors, cell suppression CMS availability matrix (796 state-years) + record layouts method, fidelity tiers, defect catalog, known truths, calibration starting points to copy

A work in progress — it will keep being updated.

Recommendations and observations welcome: bstaiger@berkeley.edu